

Governed AI Software Delivery

A governed engineering approach for large-scale distributed systems

Using domain boundaries, specifications, contracts, and AI-assisted implementation to build complex systems safely and predictably

Why This Approach

The Problem

Business logic spread across many services and repos

Ownership boundaries often unclear

Cross-service changes create hidden coupling

AI tools accelerate delivery, but also accelerate architectural drift

The Need

We need an approach that makes AI useful without losing control.

The Foundation: Specification-Driven Development (SDD)

SDD is an approach where formal specifications are written before code.
Specs define what the system should do. Code is an implementation of those specs.

What SDD Provides

- Clear requirements before implementation
- API and contract definitions up front
- Testable acceptance criteria
- Traceability from spec to code to test
- Reviewable design artifacts

SDD Workflow

- Write specification
- Review and approve specification
- Implement against specification
- Validate implementation matches spec
- Ship

SDD works well for single services and small teams. But what happens at scale?

Why SDD Alone Is Not Enough at Scale

What SDD Does Not Address

- Who owns which part of the system (domain boundaries)
- How business language maps to domain language
- How specs relate across service boundaries
- What AI is and is not allowed to change
- How to prevent AI from creating cross-domain coupling
- How to enforce architectural invariants across repos

What Goes Wrong

- Specs without domain context drift from architecture
- AI implements specs in the wrong bounded context
- AI creates shared models and hidden coupling
- Cross-service changes bypass contract discipline
- Business terms are ambiguous across teams

SDD tells you what to build. It does not tell you where it belongs, who owns it, or what rules AI must follow.

Governed AI Delivery = SDD + Domain Governance + AI Constraints

Governed AI Software Delivery keeps everything SDD provides and adds the layers needed for large-scale distributed systems.

SDD (Foundation)

- Specifications before code
- API contracts up front
- Testable acceptance criteria
- Traceability
- Reviewable artifacts

+ Domain Governance

- Bounded context ownership
- Ubiquitous language per domain
- Context maps across services
- Domain invariant protection
- Intent translation layer
- Cross-domain interaction rules

+ AI Constraints

- AI works within bounded contexts
- AI drafts, humans decide
- Governance rules limit AI scope
- Architecture violations detected
- Audit trail for AI changes
- Structured prompts from specs

SDD is the engine. Domain governance is the steering. AI constraints are the guardrails.

The Core Idea

Governed AI Software Delivery is a six-layer delivery model where:

Business intent captures stakeholder goals

Intent translation bridges business and domain language

Domain specifications define ownership and invariants

Contracts define interactions

Delivery execution combines human review with AI-assisted implementation

Verification and governance enforce rules and quality

Goal: Make AI-assisted development repeatable, reviewable, testable, and architecture-aligned.

Why "Governed" Matters

This approach is not about free-form AI coding. It is about ensuring that delivery is:

Predictable

Same inputs lead to similar outputs

Bounded

AI works within clear context limits

Verifiable

Changes map to specs, contracts, and tests

Governed

Architecture rules are enforced

Auditable

Design decisions and impacts are visible

Key principle: AI should amplify engineering discipline, not replace it.

DDD as the Control Layer

Domain-Driven Design provides the structure AI needs:

Bounded contexts

Ubiquitous language

Aggregate boundaries

Domain invariants

Context maps

Published interfaces

Why it matters for AI

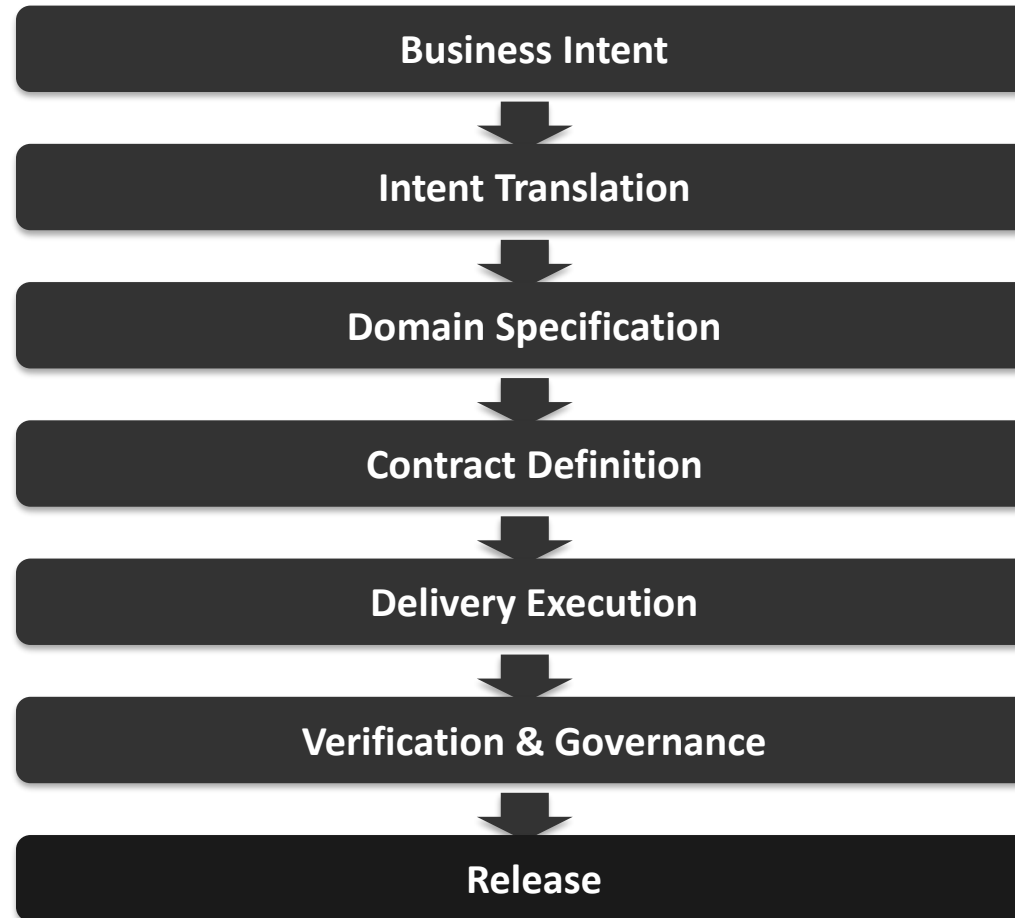
Without domain boundaries, AI tends to create shared models, hidden coupling, and accidental complexity.

DDD gives AI the constraints it needs to produce architecturally sound code.

The Six-Layer Delivery Model

1	Business Intent Layer	Epics, stakeholder asks, policy goals, operational outcomes
2	Intent Translation Layer	Business language → domain language, use cases, ownership, constraints
3	Domain Specification Layer	Bounded contexts, glossary, aggregates, invariants, context map
4	Contract Layer	REST/gRPC APIs, async events, schemas, read models
5	Delivery Execution Layer	Human design review, AI-assisted implementation, refactoring, tests
6	Verification & Governance	Contract tests, invariant tests, architecture rules, ADRs, PR review

Delivery Flow



AI does not start from a blank prompt. AI starts from a governed specification.

AI-Assisted Intent Translation (Layer 1 → 2)

Business stakeholders write in their own language — not in DDD terms.

This is where the most value gets lost. AI can draft the translation; a domain expert must validate it.

What AI Produces (Draft)

Which bounded context(s) are affected

Business terms mapped to ubiquitous language

Candidate use cases (actor, action, outcome)

Domain invariants that must hold

Cross-context dependencies

Open questions for the domain expert

What AI Cannot Do

Validate domain ownership decisions

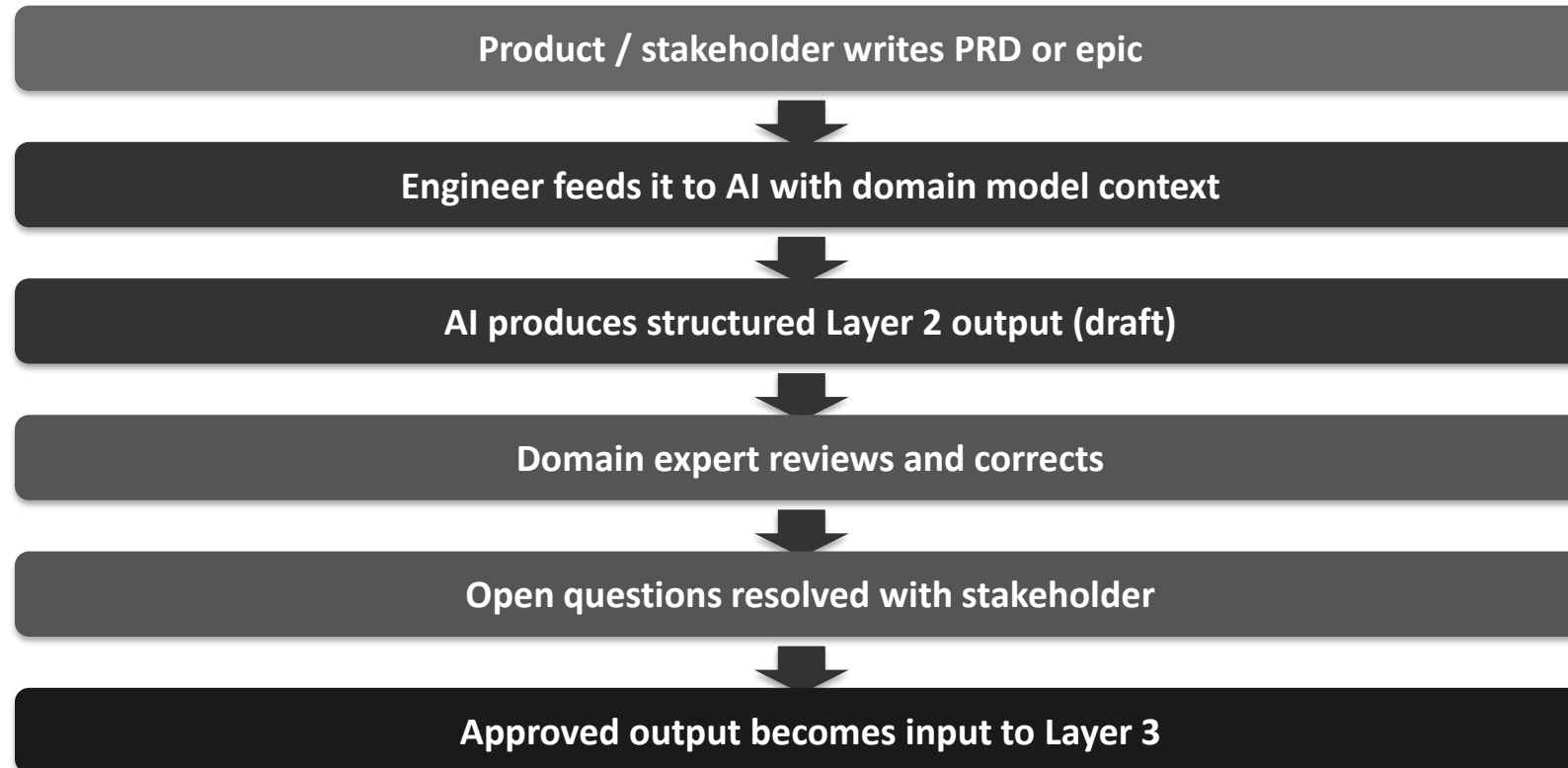
Resolve ambiguity in business language

Invent missing domain concepts

Replace domain expert review

AI is the drafter, not the decider. It turns a 2-hour decomposition into a 20-minute review.

Intent Translation Workflow



AI Prompt Structure for Intent Translation

Given this business requirement:

[paste epic / user story / PRD excerpt]

And this domain model:

[paste or reference bounded-contexts.md, ubiquitous-language.md, context-map.md]

Produce:

1. Which bounded context(s) are affected
2. Map business terms to ubiquitous language terms (flag new ones)
3. Candidate use cases (actor, action, outcome)
4. Domain invariants that must hold
5. Cross-context dependencies (if any)
6. Ownership: which team/context is primary
7. Constraints or prohibited actions
8. Open questions for the domain expert

Example requirement:

"Let customers rotate their own certificates before they expire."

AI output (excerpt):

Context: Equipment / Cert
Invariant: device must have exactly one active cert

Open Q: Does "customer" mean tenant admin or device owner?

What the AI Actually Does

- Analyze existing code and identify impacted domains
- Generate implementation plans from approved specs
- Implement changes within a bounded context
- Generate unit tests and contract tests
- Update technical documentation
- Assist refactoring with explicit constraints
- Detect architectural violations during review

AI is not the architect of record. The architecture is defined by the domain model and engineering rules.

Minimum Input Package for Each Change

Required Context

Bounded context
Business goal
Relevant use case
Domain invariants
API/event contract

Required Constraints

Acceptance criteria
Impacted repos or modules
Non-functional requirements
Prohibited actions or constraints

This turns prompting into engineering.

Suggested Project Structure

```
architecture/  
  system-context.md  
  bounded-contexts.md  
  context-map.md  
  adr/  
  
domains/  
  iam/  
    ubiquitous-language.md  
    aggregates.md  
    invariants.md  
    use-cases.md  
    api-contract.yaml  
  equipment/ | event/ | topology/
```

```
services/  
  apps/  
  packages/  
  tests/  
  
ai-rules/  
  development-rules.md  
  review-checklist.md  
  workflow-templates.md
```

This structure gives AI a stable source of truth.

Governance Rules

No cross-domain direct database access

All cross-domain interaction through APIs, events, or read models

Contracts must be updated together with implementations

Aggregate invariants must be preserved

Shared "common business model" packages are not allowed

Ambiguous ownership requires an ADR

Every change must map to tests

These rules reduce AI-induced entropy.

Fit for Large-Scale Distributed Systems

This approach is especially useful when the system has:

Many services

Many repositories

Multiple teams

Event-driven integration

Long-lived business rules

Strong security/reliability requirements

Constant change pressure

It turns a distributed system from a loosely evolving codebase into a governed delivery system.

How This Differs

Traditional Agile

Story-driven

Architecture may drift

AI use is ad hoc

Free-form AI Coding

Fast at local generation

Weak at system-level consistency

High risk of hidden coupling

Governed AI Delivery

Spec-driven

Domain-governed

Contract-aware

Test-aligned

Scalable across teams

Expected Benefits

Better architectural consistency

Safer AI-assisted delivery

Faster onboarding for engineers and agents

Clearer cross-team ownership

Improved change impact analysis

Stronger contract discipline

Better test coverage and traceability

Easier modernization of legacy systems

Net effect: Higher delivery speed with lower architectural chaos.

Risks and Pitfalls

Domain boundaries are poorly defined

Specs are too vague or too theoretical

Stories are too large and span too many contexts

Contracts are undocumented

Teams allow direct shared data access

AI is given too much freedom

Testing is weak or missing

This approach does not remove engineering work. It makes it more structured and leverageable.

Practical Rollout Plan

Phase 1 Reverse Engineer

- Identify bounded contexts
- Map service interactions
- Define domain glossary
- Document contracts

Phase 2 Pilot Domain

- Choose a bounded, valuable domain
- Define specs and governance rules
- Use AI for controlled implementation

Phase 3 Standardize

- Create templates
- Create review checklists
- Create contract validation patterns

Phase 4 Scale

- Roll out to more domains
- Align repos and pipelines
- Measure quality improvements

Good Pilot Candidates

Pick a domain that is:

- Important but not the most dangerous
- Bounded and understandable
- Testable in isolation
- Useful across teams

Good examples:

- Event search
- Device inventory read model
- Certificate rotation workflow
- Topology query service

Avoid starting with:

- Global IAM core
- Billing core
- Deeply coupled orchestration flows

Definition

Governed AI Software Delivery is a governed software engineering approach in which domain boundaries, specifications, contracts, and explicit architecture rules guide AI-assisted analysis, implementation, testing, and system evolution for large-scale distributed systems.

Bottom Line

AI should not be allowed to "figure out the system" by itself.

For large distributed systems, the winning model is:

Domain-defined

Spec-driven

Contract-governed

AI-assisted

Test-verified

That is the essence of Governed AI Software Delivery.

Governed AI Software Delivery

From:

Prompt-driven coding
Fragmented architecture
Uncontrolled acceleration

To:

Governed engineering
Scalable delivery
Predictable system evolution